

UNIVERSITY OF ASIA PACIFIC
DEPARTMENT OF CSE

Lab Report - 7

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : SULTAN RUHUL KUDDUS NAYAN

STUDENT ID : 23101159

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 2

DATE : 13-10-2026

SUBMITTED TO

ATIA RAHMAN ORTHI

LECTURER AT UAP

1. **Problem:**

Given the following disk request queue and initial disk head position:

- Request Queue: [3, 60, 10, 25, 30, 4]
- Initial Head Position: 41

Implement the following Disk Scheduling Algorithms:

- Shortest Seek Time First (SSTF)
- First Come First Serve (FCFS)

For each algorithm, calculate the total disk head movement and represent the request servicing sequence. Finally, compare the performance of SSTF and FCFS based on total head movement and determine which algorithm is more efficient for the given request queue**.**

2. Source Code

```
requests = [3, 60, 10, 25, 30, 4]
total_head_movement = 0
seek_seq = []
current_head = 41

pending_requests = list(requests)

while pending_requests:
    closest_req = min(pending_requests, key=lambda req:
abs(current_head - req))
    distance = abs(current_head - closest_req)
    total_head_movement += distance
    current_head = closest_req
    seek_seq.append(closest_req)
    pending_requests.remove(closest_req)
print("Total_head_movement:", total_head_movement)
print("Seek Sequence:", seek_seq)

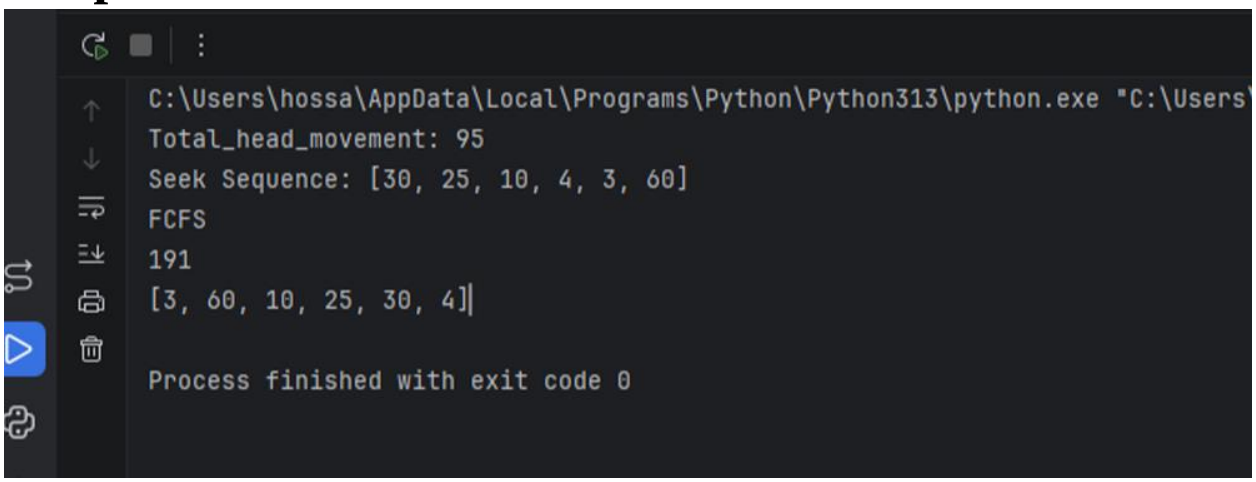
print("FCFS")

total_head_movement = 0
current_head = 41
seek_seq = []

for requests in requests:
    distance = abs(current_head - requests)
    total_head_movement += distance
    seek_seq.append(requests)
    current_head = requests

print(total_head_movement)
print(seek_seq)
```

3. Output



4. Discussion

In this experiment, the FCFS (First Come First Served) and SSTF (Shortest Seek Time First) disk scheduling algorithms were evaluated using the same set of disk requests and the same initial head position. Their performance was compared on the basis of the total head movement, measured in tracks. The results show that FCFS incurred a total head movement of 191 tracks, whereas SSTF required only 95 tracks.

This difference arises from the way each algorithm selects the next request. FCFS services requests strictly in their order of arrival, without considering how far each request is from the current head position, which often causes long and unnecessary back-and-forth movement. SSTF, in contrast, always serves the pending request closest to the current head position, thereby reducing needless head travel.

Compared with FCFS, SSTF saved 96 tracks of head movement, a reduction of approximately 50.26%. These findings demonstrate that, for the given request sequence, SSTF is considerably more efficient than FCFS in terms of total head movement.I kept your numbers the same, and they check out: $191 - 95 = 96$, and $96 / 191 \approx 50.26\%$.If you want, I can also make it shorter for a lab report conclusion, or add a note about SSTF's drawback (possible starvation of far-away requests) to give a more balanced discussion.

Conclusion :

This experiment demonstrated the working principles of the FCFS (First Come First Served) and SSTF (Shortest Seek Time First) disk scheduling algorithms and highlighted the performance differences between them. For the given input, SSTF serviced all requests with a total head movement of 95 tracks, compared with 191 tracks for FCFS, a reduction of approximately 50.26%. SSTF therefore proved to be the more efficient algorithm in terms of seek distance.

However, this efficiency comes with a trade-off. Because SSTF always favors the nearest pending request, requests located far from the current head position may wait for a long time or even suffer starvation. FCFS, although less efficient, services requests in a fair and predictable order.

Hence, the choice of a disk scheduling algorithm depends on the system's priorities: SSTF is preferable when minimizing head movement is the main goal, while FCFS (or a fairer algorithm such as SCAN) is better when fair request servicing is the primary concern.

GitHub: https://github.com/sultan36912/Operating-System-Lab..git